

The BASIC User's Guide

A guide to this GW-BASIC dialect and its full-screen editor. It assumes you have written BASIC in some dialect before, and concentrates on how this one behaves — the type system, the editor, the I/O statements, and the places where classic Microsoft BASIC works differently from what you might expect.

Contents

1. Getting Started
 2. The Screen Editor
 3. Language Fundamentals
 4. Numbers and Expressions
 5. Strings
 6. Control Flow
 7. Arrays and DATA
 8. User-Defined Functions
 9. Console Output: PRINT and PRINT USING
 10. Keyboard Input
 11. Screen Control
 12. File I/O
 13. Error Handling
 14. Managing Programs and the Workspace
 15. Differences from Real GW-BASIC
 16. Appendix A: Error Codes
 17. Appendix B: Editor Key Reference
-

1. Getting Started

Build and run:

```
make                # produces ./basic
./basic             # full-screen editor (the normal way to use it)
./basic DEMO.BAS    # editor with a program already loaded – type RUN
./basic -r DEMO.BAS # run a program headless, no editor, then exit
```

You can also pipe commands in, which is handy for scripting and testing:

```
$ echo 'PRINT 2+2' | ./basic
4
Ok
```

Start `./basic` and you are at the **Ok** prompt, exactly like an old home computer. Anything you type either runs immediately or, if it starts with a line number, becomes part of the stored program:

```
10 FOR I=1 TO 3
20 PRINT "HELLO";I
30 NEXT
RUN
HELLO 1
HELLO 2
HELLO 3
Ok
```

`?` is shorthand for `PRINT`, so `?2+2` works at the prompt and inside programs. Keywords are case-insensitive: `print`, `PRINT` and `Print` are the same.

To leave, type `SYSTEM`.

2. The Screen Editor

The editor works like a Commodore 64: **the screen itself is the editor**. There is no separate editing mode — the cursor goes anywhere, you change what you see, and pressing **Enter** "commits" the line under the cursor.

- If the committed line starts with a number, it is stored in the program (replacing any existing line with that number).
- If it is just a number, that program line is **deleted**.
- Anything else executes immediately, then `Ok` is printed.

Editing a listed line in place

This is the signature move. Suppose you have:

```
10 PRINT "HELLO"
```

Type `LIST`, cursor **up** onto the listed line, cursor **right** to the character you want, type over it, press **Enter**. The modified line is re-committed to the program. `LIST` again to confirm.

Two things to know:

- A logical line may wrap across screen rows (long lines, or `WIDTH 40`). The editor knows which rows belong together and commits the whole thing.
- The screen remembers **everything**, including old output. If you press Enter on a row containing stale text, that text is what gets committed — the classic C64 gotcha. (Typing `LIST` over a row that says `20 FOR I=...` commits `LISTOR I=...`, which is a syntax error. Authentic!)

Keys

Key	Action
Arrows	Move the cursor anywhere on screen
Enter	Commit the logical line under the cursor
Backspace	Delete left of cursor, shift the row left
Delete	Delete at cursor
Insert	Toggle insert/overtyping (Enter resets to overtype)
Esc	Erase the current logical line
Home / End	Top-left of screen / end of current line
Tab	Spaces to the next 8-column stop
Ctrl-L	Clear the screen
Ctrl-C	Interrupt a running program (<code>Break in n</code>); cancel AUTO
F1-F10	Soft keys (below)

Soft keys

F1-F10 type predefined text: F1 is `LIST`, F2 is `RUN +Enter`, F3 `LOAD`, F4 `SAVE`, F5 `CONT +Enter`, F7 `TRON`, F8 `TROFF`. Redefine them with `KEY n, "text"` (a trailing `CHR$(13)` presses Enter for you), list them with `KEY LIST`, and show them on the bottom screen row with `KEY ON` / hide with `KEY OFF`:

```
KEY 2, "RUN" + CHR$(13)
KEY ON
```

AUTO — automatic line numbering

```
AUTO          ' numbers from 10 in steps of 10
AUTO 100,5    ' from 100 in steps of 5
```

The editor types each line number for you; you supply the rest and press Enter. If the line already exists, the prompt shows an asterisk (`120*`) with the cursor on it — press Enter to keep the existing line and move on, or type over it to replace. Press Enter on an empty new-line prompt, or Ctrl-C, to leave AUTO mode.

EDIT — jump to a line

`EDIT 130` lists line 130 and parks the cursor on it, ready to modify. `EDIT .` edits the most recently entered or listed line.

Interrupting and resuming

Ctrl-C stops a running program with `Break in <line>`. `CONT` resumes it — variables intact — as does resuming after `STOP`. If you edit the program in between, `CONT` reports `Can't continue.`

3. Language Fundamentals

Lines and statements

A program is numbered lines, 0-65529. Multiple statements on a line are separated by colons:

```
10 X=1: Y=2: PRINT X+Y
```

Comments are REM or an apostrophe; everything after them is ignored:

```
20 REM this is a comment
30 PRINT "HI" ' so is this
```

Variables and types

Variable names start with a letter and may contain letters, digits and periods (TOTAL , X9 , NET.PAY). The last character may be a **type suffix**:

Suffix	Type	Range / size
%	Integer	16-bit, -32768..32767
!	Single precision	~7 significant digits (the default)
#	Double precision	~16 significant digits
\$	String	0-255 characters

A% , A! , A# and A\$ are **four different variables**. An unsuffixed name defaults to single precision, unless a DEFINT / DEFSNG / DEFDBL / DEFSTR statement says otherwise for its first letter:

```
10 DEFINT I-N ' FORTRAN-style: I..N default to integer
20 DEFSTR S ' names starting with S default to string
30 S = "no dollar sign needed"
```

An explicit suffix always wins over a DEFxxx rule.

Constants

```
100      decimal integer
3.14     single precision
1.5E10   single precision (E exponent)
1.5D10   double precision (D exponent)
123456789 more than 7 digits: automatically double
3.14#    suffixes force a type: # double, ! single, % integer
&HFF     hexadecimal (255); &H8000 is -32768 (16-bit wrap)
&017, &17 octal (15)
"text"   string; the closing quote may be omitted at end of line
```

How keywords are recognized (read this!)

Like Microsoft BASIC-80, a keyword is only recognized when it is **not followed by a letter, \$ or .** . The practical consequences:

TOTAL = 5	' fine: TOTAL is a variable (T0 is not matched)
FOR I=1T010	' fine: digits may touch keywords
GOT010	' fine
FORI = 7	' FORI is a VARIABLE – this is an assignment!
PRINTX	' a variable named PRINTX, not PRINT X

So you can squeeze out spaces between keywords and numbers, but a keyword must not run into a following word. When in doubt, use spaces.

Reserved words cannot be variable names (`PRINT=5` is a syntax error), and names beginning with `FN` are reserved for user functions.

4. Numbers and Expressions

Operators, highest precedence first

Operators	Meaning
<code>^</code>	Exponentiation (left-associative : $2^3^2 = 64$)
<code>-</code>	Unary minus ($-2^2 = -4$: <code>^</code> binds tighter)
<code>*</code> <code>/</code>	Multiply, divide
<code>\</code>	Integer division (operands rounded to 16-bit, result truncated)
<code>MOD</code>	Integer remainder (sign follows the dividend)
<code>+</code> <code>-</code>	Add, subtract (<code>+</code> also concatenates strings)
<code>=</code> <code><></code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	Comparison: result is -1 (true) or 0 (false)
<code>NOT</code>	Bitwise complement
<code>AND</code> , <code>OR</code> , <code>XOR</code> , <code>EQV</code> , <code>IMP</code>	Bitwise on 16-bit integers

Because true is -1 (all bits set), the logical operators double as bitwise operators, and you can write things like:

```
10 IF A > 0 AND A < 10 THEN PRINT "single digit"
20 PRINT 6 AND 3      ' prints 2 (bitwise)
30 FLAG = (X > 5)     ' FLAG is -1 or 0
```

The operands of `\`, `MOD` and the logical operators must fit in 16 bits; they are rounded first: $7.5 \setminus 2$ is 4 (7.5 rounds to 8).

Arithmetic that doesn't stop your program

This dialect reproduces GW-BASIC's soft arithmetic errors. Division by zero and floating-point overflow print a message, substitute **machine infinity** ($\pm 1.701412\text{E}+38$), and carry on:

```
10 PRINT 1/0
20 PRINT "still running"
RUN
Division by zero
1.701412E+38
still running
```

Likewise, 16-bit integer arithmetic that overflows is silently redone in single precision: $30000 + 30000$ is 60000 , not an error. But an assignment out of range is a real error: `A% = 60000` stops with `overflow`. These soft errors cannot be trapped with `ON ERROR`.

Rounding rules

- `CINT(x)` rounds to the nearest integer, **half to even**: `CINT(2.5) = 2`, `CINT(3.5) = 4`. The same rule applies when storing to a `%` variable.

- `INT(x)` floors: `INT(-2.5) = -3`.
- `FIX(x)` truncates toward zero: `FIX(-2.5) = -2`.
- `PRINT USING` rounds half **away** from zero (see chapter 9).

How numbers print

Positive numbers print with a leading space (where the minus sign would be), and every number gets a trailing space:

```
PRINT 1; -2; 3
1 -2 3
```

Singles show up to 7 significant digits, doubles 16. Values smaller than 0.01 or too wide for fixed notation switch to scientific, with `E` (single) or `D` (double) exponents: `1E+07`, `1.234567890123457D+18`. There is no leading zero before a bare decimal point: `.5`, `-.25`.

Math functions

`ABS` `SGN` `SQR` `EXP` `LOG` `SIN` `COS` `TAN` `ATN` (radians), plus the conversions `CINT` `CSNG` `CDBL` `INT` `FIX`. `SQR(-1)` and `LOG(0)` raise `Illegal function call`; `EXP(1000)` overflows softly to machine infinity.

Random numbers

`RND` returns the next value in a deterministic sequence — the **same sequence on every run** unless you seed it:

```
10 RANDOMIZE TIMER          ' seed from the clock: different every run
20 PRINT INT(RND*6)+1       ' a die roll
```

`RND(0)` repeats the last value; `RND` with a negative argument reseeds deterministically. Bare `RANDOMIZE` prompts for a seed (`-32768..32767`). `TIMER` is seconds since midnight, with fractions; `DATE$` and `TIME$` return `"MM-DD-YYYY"` and `"HH:MM:SS"`.

5. Strings

Strings hold 0–255 bytes. Exceeding 255 (for example by concatenation) raises `String too long`.

```
10 A$ = "FOO" + "BAR"
20 PRINT LEN(A$); A$
6 FOOBAR
```

Comparisons are byte-by-byte ASCII, so `"abc" > "ABC"` and a prefix sorts first (`"AB" < "ABC"`).

The function toolbox

Function	Result
<code>LEFT\$(s\$,n)</code> / <code>RIGHT\$(s\$,n)</code>	First / last n characters
<code>MID\$(s\$,p[,n])</code>	Substring from position p (1-based)
<code>INSTR([start,]s\$,t\$)</code>	Position of t\$ in s\$, 0 if absent
<code>LEN(s\$)</code>	Length
<code>ASC(s\$)</code> / <code>CHR\$(n)</code>	First byte's code / one-character string
<code>STRING\$(n,c)</code>	n copies of a character (code or string)
<code>SPACE\$(n)</code>	n spaces
<code>VAL(s\$)</code>	Numeric value; blanks are ignored anywhere : <code>VAL("1 2") = 12</code>
<code>STR\$(x)</code>	Number as string, with the leading sign space: <code>STR\$(5) = " 5"</code>
<code>HEX\$(n)</code> / <code>OCT\$(n)</code>	Hex / octal, 16-bit wrap: <code>HEX\$(-1) = "FFFF"</code>

MID\$ as a statement

`MID$` on the left side of `=` replaces characters **in place** — the string never changes length:

```
10 A$ = "KANSAS CITY, MO"
20 MID$(A$, 14) = "KS"
30 PRINT A$
KANSAS CITY, KS
```

An optional third argument caps how many characters are copied.

6. Control Flow

IF / THEN / ELSE

All on one line. The branches can hold multiple statements; `ELSE` binds to the nearest `IF`:

```
10 IF X > 0 THEN PRINT "POS": C=C+1 ELSE PRINT "NOT POS"
20 IF X = 1 THEN 100           ' THEN <line> is a GOTO
30 IF X = 2 GOTO 200          ' so is IF...GOTO
40 IF A THEN IF B THEN PRINT "AB" ELSE PRINT "A" ELSE PRINT "NOT A"
```

Any nonzero condition value is true.

FOR / NEXT

```
10 FOR I = 10 TO 1 STEP -3
20 PRINT I;
30 NEXT I
   10  7  4  1
```

Details that matter:

- The limit and step are evaluated **once**, at loop entry.
- If the initial value is already past the limit, the body is **skipped entirely** (the variable keeps its initial value).
- After normal completion the variable has gone one step past: `FOR I=1 TO 3` leaves `I = 4`.
- `NEXT` may name its variable, close several loops (`NEXT J, I`), or be bare (closes the innermost loop).
- Starting a new `FOR` with the same variable discards the older loop.

WHILE / WEND

```
10 WHILE BALANCE < 1000
20 BALANCE = BALANCE * 1.05 : YEARS = YEARS + 1
30 WEND
```

The matching `WEND` must appear **after** the `WHILE` in the program text — the interpreter scans for it on every iteration, exactly like GW-BASIC, and raises `WHILE without WEND` if it isn't there.

GOSUB / RETURN

```
10 GOSUB 1000
20 PRINT "back": END
1000 PRINT "in subroutine"
1010 RETURN
```

`RETURN <line>` returns to a specific line instead of the call site. GW-BASIC's unified stack semantics are reproduced: a `RETURN` throws away any `FOR` or `WHILE` loops the subroutine left

open, and a subroutine's loops cannot collide with the caller's (a `FOR I` inside a subroutine does not disturb a `FOR I` in the caller).

Computed jumps

```
10 INPUT "Choice 1-3"; C
20 ON C GOSUB 100, 200, 300
```

If the value is 0 or larger than the list, execution simply falls through.

Timer interrupts

`ON TIMER(n) GOSUB line` arms a software interrupt that fires every `n` seconds while the program runs, once enabled with `TIMER ON`:

```
10 ON TIMER(2) GOSUB 500
20 TIMER ON
30 GOTO 30          ' busy doing "work"
500 PRINT "tick": RETURN
```

`TIMER OFF` disables it. The trap will not re-fire until the handler has `RETURN`ed.

Stopping

`END` finishes silently (and closes files); `STOP` prints `Break in <line>`. Both can be resumed from the prompt with `CONT`.

7. Arrays and DATA

Arrays

```
10 DIM A(20), T$(5,3)      ' explicit dimensions
20 A(0) = 1: A(20) = 99
```

Subscripts run from 0 to the declared maximum — `DIM A(20)` has 21 elements. `OPTION BASE 1` makes them start at 1 (it must appear before any array exists). Using an undimensioned array implicitly dimensions it to 10 in each subscript. `DIM`ming an existing array is a `Duplicate Definition`; `ERASE A` deletes it so it can be re-dimensioned. Arrays and scalars are separate: `A` and `A(1)` can coexist.

`SWAP X, Y` exchanges two variables of the same type (both must already have values).

DATA, READ, RESTORE

```
10 DATA "Smith, John", 42, 3.9
20 DATA Jones,17,2.2
30 FOR I = 1 TO 2
40   READ N$, AGE, GPA
50   PRINT N$; AGE; GPA
60 NEXT
70 RESTORE 20          ' rewind the pointer (optionally to a line)
80 READ X$: PRINT X$
```

Unquoted items have surrounding blanks trimmed; quote an item when it must contain commas or colons. Empty items (`DATA 1,,3` — or a bare `DATA`) read as 0 or `"`. Reading a non-numeric or quoted item into a numeric variable is a `Syntax error` reported **at the DATA line**. Running out raises `Out of DATA`.

8. User-Defined Functions

`DEF FN` defines a one-line function. It must appear in the program (it is `Illegal direct` at the prompt) and must execute before its first use:

```
10 DEF FNAREA(R) = 3.141593 * R * R
20 DEF FNSHOUT$(S$) = S$ + "!"
30 PRINT FNAREA(2); FNSHOUT$("HEY")
    12.56637 HEY!
```

The type suffix on the name sets the return type. Parameters shadow global variables of the same name only during the call. Functions may call other `FN` functions; runaway recursion is stopped with `Out of memory`.

9. Console Output: PRINT and PRINT USING

PRINT

- `;` places items side by side (numbers already carry their spaces).
- `,` advances to the next 14-column **print zone** — a quick way to make columns.
- A trailing `;` or `,` suppresses the newline.
- `TAB(n)` moves to column n (down a line if already past it); `SPC(n)` emits n spaces.

```
10 PRINT "NAME", "QTY"
20 PRINT "BOLT", 41
30 PRINT "X"; TAB(20); "Y"
NAME          QTY
BOLT          41
X              Y
```

`WRITE` is `PRINT`'s machine-readable cousin: it quotes strings and separates items with commas — exactly the format `INPUT #` reads back:

```
WRITE "Smith, John", 42
"Smith, John",42
```

PRINT USING

`PRINT USING "format"; value [; value...]` formats values into a template. Numeric field characters:

Spec	Meaning	Example (value)	Output
<code>#</code>	One digit position, right-justified	<code>"####"; 12</code>	12
<code>.</code>	Decimal point; rounds half away from zero	<code>"##.##"; 2.5</code>	2.50
<code>,</code>	Comma grouping (and one extra column)	<code>"##,###"; 12345</code>	12,345
<code>+ leading</code>	Print the sign	<code>"#####"; 5</code>	+5
<code>+ / - trailing</code>	Sign after the number	<code>"###-"; -5</code>	5-
<code>**</code>	Fill with asterisks (adds 2 columns)	<code>"***###.##"; 12.3</code>	***12.3
<code>\$\$</code>	Floating dollar sign (adds 2 columns)	<code>"\$\$\$###.##"; 7.5</code>	\$7.50
<code>**\$</code>	Both	<code>"***\$###.##"; 7.5</code>	***\$7.50
<code>^^^</code>	Scientific notation	<code>"##.##^"; 234.56</code>	2.35E+02
<code>^^^^</code>	...with a 3-digit exponent	<code>"#.##^"; 123</code>	.1E+003

If a number doesn't fit its field it is printed anyway with a `%` prefix (`"###"; 123` gives `%123`). At most 24 digit positions per field. With no digit position left of the point, the leading zero is dropped: `"..###"; .555` prints `.56`.

String fields: `!` takes the first character, `&` the whole string, and `\ \` (backslashes with n spaces between) a fixed n+2 characters, left-justified. Any other character in the template

prints literally; `\` escapes the next character. The template recycles if there are more values than fields:

```
10 PRINT USING "ITEM ! COSTS $$.##.##"; "WIDGET", 4.5  
ITEM W COSTS    $4.50
```

`LPRINT` and `LLIST` work like `PRINT / LIST` but append to the file `printer.out` instead of a printer.

10. Keyboard Input

INPUT

```
10 INPUT "How many"; N      ' prompts: How many?
20 INPUT "Name: ", N$      ' comma: no question mark added
30 INPUT X, Y, Z           ' expects: 1,2,3
```

Replies are comma-separated; quote a string reply that itself contains commas. If the reply doesn't match (wrong count, text where a number is expected), you get `?Redo from start` and the statement asks again. Pressing Enter on an empty line **keeps the variables' previous values**. `INPUT;` (semicolon right after the keyword) leaves the cursor on the same line afterwards. `INPUT` is not allowed at the Ok prompt (`Illegal direct`).

LINE INPUT

Reads an entire raw line — commas, quotes and all — into one string variable:

```
10 LINE INPUT "Address: "; A$
```

INKEY\$ — polling the keyboard

`INKEY$` returns one pending keystroke, or `""` immediately if none is waiting. It never echoes and never blocks — perfect for game loops:

```
10 K$ = INKEY$
20 IF K$ = "" THEN 10
30 IF K$ = CHR$(27) THEN END      ' Esc
40 IF LEN(K$) = 2 THEN PRINT "extended key"; ASC(RIGHT$(K$,1)): GOTO 10
50 PRINT "you typed "; K$: GOTO 10
```

Ordinary keys arrive as one character (Enter is `CHR$(13)`). Special keys arrive as two: `CHR$(0)` followed by the scan code — arrows are 72/80/75/77 (up/down/left/right), Home 71, End 79, F1–F10 are 59–68. Ctrl-C is never returned: it always interrupts the program.

`INPUT$(n)` waits for exactly `n` keystrokes without echo:

```
10 PRINT "Press any key": X$ = INPUT$(1)
```


11. Screen Control

The screen is 80×24 text (or whatever your terminal provides), with the classic 16 foreground and 8 background colors.

```
10 CLS                ' clear screen (CLS 0/1/2 also accepted)
20 COLOR 14, 1        ' yellow on blue
30 LOCATE 5, 10       ' row 5, column 10 (1-based)
40 PRINT "Hello at 5,10";
50 COLOR 7, 0         ' back to white on black
```

- `LOCATE row, col [, cursor]` — the third argument shows/hides the cursor.
- `POS(0)` and `CSRLIN` return the current column and row.
- `SCREEN(r, c)` returns the character code at a screen position; `SCREEN(r, c, 1)` returns its color attribute (foreground + 16×background).
- `WIDTH 40 / WIDTH 80` switch the logical line width (clears the screen).
- `VIEW PRINT 5 TO 20` confines printing and scrolling to rows 5–20 — useful for a fixed status header. Bare `VIEW PRINT` releases it.
- `CHR$` codes 128–255 (and 1–31 when printed) are the IBM CP437 set, so box-drawing characters work: `PRINT STRING$(20, 205)` draws a double rule. Control codes do the classic things: 7 beeps, 11 homes the cursor, 12 clears the screen, 28–31 nudge the cursor.

A tiny status-line demo:

```
10 CLS : VIEW PRINT 2 TO 24
20 LOCATE 1, 1 : COLOR 0, 7 : PRINT SPACE$(79); : COLOR 7, 0
30 LOCATE 1, 2 : COLOR 0, 7 : PRINT "*** REPORT ***"; : COLOR 7, 0
40 LOCATE 2, 1
50 FOR I = 1 TO 50 : PRINT "line"; I : NEXT      ' header stays put
```

`BEEP` beeps. `SOUND` and `PLAY` accept their arguments but just beep — there is no tone generator in a terminal.

12. File I/O

Files are referenced by a number, 1–15, attached with `OPEN` and released with `CLOSE` (or `RESET`, which closes everything). Two `OPEN` spellings are accepted:

```
OPEN "data.txt" FOR INPUT AS #1      ' modern form
OPEN "I", #1, "data.txt"            ' classic form (I, O, A, R)
```

Modes: `INPUT` (read), `OUTPUT` (create/truncate), `APPEND`, `RANDOM` (read/write records — the default). Opening a missing file for `INPUT` is `File not found`; opening a file that is already open (or `KILL/NAME` on one) is `File already open`.

Sequential files

```
10 OPEN "log.txt" FOR OUTPUT AS #1
20 PRINT #1, "plain line"
30 WRITE #1, "Smith, John", 42      ' quoted+comma format
40 CLOSE #1
50 OPEN "log.txt" FOR INPUT AS #1
60 LINE INPUT #1, L$                ' whole line
70 INPUT #1, N$, AGE                ' parses WRITE format back
80 PRINT L$: PRINT N$: AGE
90 IF NOT EOF(1) THEN PRINT "more..."
100 CLOSE #1
```

`INPUT #` reads comma- or newline-separated items; numbers also end at a blank, so `1.5 2.5 3.5` on one line reads as three numbers. `EOF(n)` is true at end of file; `LOF(n)` is the size in bytes; `LOC(n)` the rough position; `INPUT$(n, #f)` reads `n` raw bytes.

Random-access files

A random file is an array of fixed-size records on disk. `FIELD` overlays string variables onto the record buffer; `LSET` / `RSET` fill them (left- or right-justified); `PUT` and `GET` move the buffer to and from a numbered record:

```
10 OPEN "people.dat" FOR RANDOM AS #1 LEN = 32
20 FIELD #1, 20 AS NM$, 4 AS AGE$, 8 AS BAL$
30 LSET NM$ = "BABBAGE"
40 LSET AGE$ = MKS$(33)              ' numbers must be packed
50 LSET BAL$ = MKD$(1234.56)
60 PUT #1, 1                        ' write record 1
70 GET #1, 1                        ' read it back
80 PRINT NM$; CVS(AGE$); CVD(BAL$)
90 CLOSE #1
```

`MKI$` / `MKS$` / `MKD$` pack an integer/single/double into 2/4/8 bytes; `CVI` / `CVS` / `CVD` unpack. Several `FIELD` statements on the same file overlay one another, so you can view one record in different layouts. Important: use `LSET` / `RSET`, not plain `LET`, to put data into a field — the **buffer** is what `PUT` writes.

Housekeeping

```
FILES          ' directory listing (FILES "*.BAS" filters)
KILL "old.dat"  ' delete (wildcards allowed)
NAME "a.dat" AS "b.dat"
SHELL "ls -l"   ' run a shell command and come back
```

13. Error Handling

Trap runtime errors with `ON ERROR GOTO`; inspect `ERR` (code) and `ERL` (line); leave the handler with `RESUME`:

```
10 ON ERROR GOTO 1000
20 INPUT "File"; F$
30 OPEN F$ FOR INPUT AS #1
40 PRINT "opened ok": CLOSE: END
1000 IF ERR = 53 THEN PRINT "No such file, try again": RESUME 20
1010 ON ERROR GOTO 0          ' anything else: make it fatal
```

- `RESUME` retries the statement that failed; `RESUME NEXT` continues after it; `RESUME <line>` jumps.
- `ON ERROR GOTO 0` disables trapping; inside a handler it re-raises the pending error as fatal, reported at the original line.
- An error inside the handler is always fatal, and a program that simply ends while inside a handler raises `No RESUME`.
- `ERROR n` raises any error on demand — useful for testing handlers or defining your own codes (codes without standard text print `Unprintable error`).

Untrapped errors stop the program with `<message> in <line>`. Note that division by zero and floating overflow are warnings, not trappable errors (chapter 4).

14. Managing Programs and the Workspace

LIST	' whole program	LIST 100-200	' a range
LIST 50-	' from 50 on	LIST .	' current line
DELETE 30-90	' remove lines		
RENUM	' renumber 10,20,... fixing up GOTO/GOSUB/THEN/ELSE/		
RENUM 100,50,5	' new start 100, from old line 50, step 5		
NEW	' erase program and variables		
CLEAR	' erase variables only (also closes files)		

Saving and loading

SAVE "GAME"	' writes GAME.BAS (plain ASCII)
LOAD "GAME"	' replaces the program (clears variables)
LOAD "GAME", R	' load and run, keeping open data files
RUN "GAME"	' shorthand: load and run
MERGE "SUBS"	' overlay lines into the current program

Chaining programs

A program can hand control to another, passing selected variables:

```
10 COMMON N$, SCORE      ' declare what survives the hop
20 N$ = "ADA": SCORE = 4200
30 CHAIN "part2"          ' loads PART2.BAS and runs it
```

CHAIN "f", 500 starts at line 500; CHAIN "f", , ALL passes every variable; CHAIN MERGE "overlay" merges instead of replacing.

Debugging and odds and ends

- TRON / TROFF — trace mode; prints [10][20]... as lines execute.
- FRE(0) — free memory (a nostalgic constant here).
- PEEK(a) / POKE a, v — a private 64 KB sandbox; DEF SEG is accepted.
- ENVIRON\$("HOME") reads environment variables; ENVIRON "X=Y" sets one.
- LPRINT / LLIST append to printer.out.
- SYSTEM exits to the shell.

15. Differences from Real GW-BASIC

This is a faithful dialect, not a perfect emulator. Knowingly different:

- **No pixel graphics.** `PSET`, `LINE` (drawing), `CIRCLE`, `DRAW`, `PAINT`, `SCREEN` modes 1+, palettes, and the light-pen/joystick/port statements raise error 73, `Advanced Feature`. The text screen (chapter 11) is fully functional.
 - **Sound is a beep.** `BEEP` works; `SOUND` and `PLAY` parse their arguments and beep once. Event traps other than `ON TIMER` raise 73.
 - Programs are stored and `SAVE` d as **plain ASCII**; tokenized binary `.BAS` files can't be loaded, and `LIST` shows lines exactly as typed.
 - `MKS$ / MKD$ / CVS / CVD` use IEEE floats, not Microsoft Binary Format — data files with packed numbers aren't byte-compatible with real GW files.
 - `PRINT # / WRITE # / INPUT #` are refused on `RANDOM`-mode files (GW allowed them through the `FIELD` buffer); use `FIELD / LSET / GET / PUT`.
 - `LET` on a `FIELD`ed string doesn't permanently detach it from the buffer: the buffer keeps the `LSET` value, but a later `GET` refreshes the variable again.
 - Numbers smaller than 0.01 always print in scientific notation (real GW's threshold differs slightly in places).
 - `LPRINT / LLIST` go to the file `printer.out`. `PEEK / POKE` touch a sandbox, not real memory; `CALL`, `USR`, `WAIT`, `INP / OUT` are stubs or error 73. There is no 64 KB memory ceiling; `FRE` is decorative.
 - `Ctrl-C` plays the role of `Ctrl-Break`.
-

Appendix A: Error Codes

#	Message	#	Message
1	NEXT without FOR	22	Missing operand
2	Syntax error	26	FOR without NEXT
3	RETURN without GOSUB	29	WHILE without WEND
4	Out of DATA	30	WEND without WHILE
5	Illegal function call	50	FIELD overflow
6	Overflow	52	Bad file number
7	Out of memory	53	File not found
8	Undefined line number	54	Bad file mode
9	Subscript out of range	55	File already open
10	Duplicate Definition	58	File already exists
11	Division by zero	61	Disk full
12	Illegal direct	62	Input past end
13	Type mismatch	63	Bad record number
14	Out of string space	64	Bad file name
15	String too long	66	Direct statement in file
17	Can't continue	67	Too many files
18	Undefined user function	70	Permission Denied
19	No RESUME	73	Advanced Feature
20	RESUME without error	75	Path/File Access Error
21-76	(others as in GW-BASIC)	76	Path not found

Trap any of them with `ON ERROR GOTO ;` ; raise them with `ERROR n` .

Appendix B: Editor Key Reference

Arrows	move cursor (the whole screen is editable)		
Enter	commit the logical line under the cursor		
Backspace	delete left	Delete	delete at cursor
Insert	toggle insert/overtyp	Esc	erase current line
Home	top-left	End	end of current line
Tab	next 8-column stop	Ctrl-L	clear screen
Ctrl-C	break program / cancel AUTO		
F1-F10	soft keys (KEY n,"..." to redefine, KEY LIST to show, KEY ON/OFF for the bottom label row)		

Prompt commands cheat sheet:

RUN [line]	CONT	LIST [range]	AUTO [start][,inc]
EDIT line	DELETE range	RENUM [n[,old[,inc]]]	
SAVE "F"	LOAD "F"[,R]	MERGE "F"	CHAIN [MERGE] "F"
NEW	CLEAR	FILES [pat]	KILL "F"
TRON / TROFF	SHELL "cmd"	WIDTH 40 80	SYSTEM